

3. События и состояние: `useState`

Цель занятия

На прошлых занятиях мы разобрали архитектуру проекта, JSX и компоненты. Теперь переходим к интерактивности: пользователь нажимает кнопку, а интерфейс меняется.

В текущем проекте уже есть готовый пример интерактивности - кнопка счетчика:

```
javascript
const [count, setCount] = useState(0)
```

и:

```
javascript
<button
  type="button"
  className="counter"
  onClick={() => setCount((count) => count + 1)}
>
  Count is {count}
</button>
```

На этом занятии мы подробно разберем, как это работает.

После занятия должны уметь:

- объяснить, что такое событие в интерфейсе;
- использовать `onClick` в JSX;
- понимать разницу между передачей функции и вызовом функции;
- объяснить, зачем компоненту нужно состояние;
- использовать хук `useState`;
- читать запись `const [count, setCount] = useState(0);`
- обновлять состояние по предыдущему значению;
- понимать, почему React обновляет интерфейс после `setCount`.

1. Что говорит документация React

В актуальной документации React события описываются как реакции на действия пользователя: клик, наведение, ввод текста, фокус на поле формы и другие взаимодействия.

Состояние в React описывается как память компонента. Компоненту часто нужно что-то запомнить между обновлениями интерфейса: число кликов, введенный текст, выбранную вкладку, открыто или закрыто меню.

Коротко:

```
text
Событие отвечает на действие пользователя.
Состояние хранит данные, которые должны влиять на интерфейс.
```

Пример:

```
javascript
function Counter() {
  const [count, setCount] = useState(0)

  return (
    <button onClick={() => setCount(count + 1)}>
      Count is {count}
    </button>
  )
}
```

Когда пользователь нажимает кнопку, вызывается обработчик события `onClick`. Он меняет состояние через `setCount`. После этого React заново отображает компонент с новым значением `count`.

2. Что такое событие

Событие - это действие, которое происходит на странице.

Примеры событий:

```
text
click - пользователь нажал на элемент
input - пользователь вводит текст
submit - пользователь отправил форму
mouseover - пользователь навел мышь
focus - пользователь поставил курсор в поле ввода
```

В обычном HTML можно встретить такую идею:

```
html
<button onclick="alert('Привет')">Нажми</button>
```

В React мы пишем обработчики событий в JSX немного иначе:

```
javascript
<button onClick={() => alert('Привет')}>
  Нажми
</button>
```

Обратите внимание:

- в React пишется `onClick`, а не `onclick`;
- обработчик передается в фигурных скобках `{}`;
- значением `onClick` должна быть функция.

3. Обработчик события

Обработчик события - это функция, которая запускается, когда событие произошло.

Пример:

```
javascript
function App() {
  function handleClick() {
    alert('Кнопка нажата')
  }

  return (
    <button onClick={handleClick}>
      Нажми
    </button>
  )
}
```

Здесь:

- `handleClick` - функция-обработчик;
- `onClick={handleClick}` - мы передаем функцию кнопке;
- **React вызовет `handleClick`**, когда пользователь нажмет кнопку.

Можно записать обработчик прямо внутри JSX:

```
javascript
<button onClick={() => alert('Кнопка нажата')}>
  Нажми
</button>
```

Для коротких действий это удобно. Для более длинной логики лучше создавать отдельную функцию с понятным названием.

4. Важно: передать функцию, а не вызвать ее сразу

Одна из самых частых ошибок новичков - случайно вызвать функцию во время рендера.

Правильно:

```
javascript
<button onClick={handleClick}>
  Нажми
</button>
```

Неправильно:

```
javascript
<button onClick={handleClick()}>
  Нажми
</button>
```

В первом случае мы передаем функцию. React вызовет ее позже, когда пользователь нажмет кнопку.

Во втором случае мы вызываем функцию сразу, когда компонент отображается. Результат вызова попадет в `onClick`, а это почти всегда ошибка.

То же правило работает со стрелочной функцией:

```
javascript
  <button onClick={() => handleClick()}>
    Нажми
  </button>
```

Здесь в `onClick` передается новая функция. Она вызовет `handleClick` только после клика.

5. Разбор кнопки из проекта

В `src/App.jsx` есть кнопка:

```
javascript
  <button
    type="button"
    className="counter"
    onClick={() => setCount((count) => count + 1)}
  >
    Count is {count}
  </button>
```

Разберем по частям.

```
javascript
  type="button"
```

Говорит браузеру, что это обычная кнопка. Это особенно важно внутри форм, чтобы кнопка случайно не отправляла форму.

```
javascript
  className="counter"
```

Подключает CSS-класс `.counter` из `App.css`.

```
javascript
  onClick={() => setCount((count) => count + 1)}
```

Говорит React: когда пользователь нажмет кнопку, нужно выполнить эту функцию.

```
javascript
  Count is {count}
```

Выводит текущее значение переменной `count` в интерфейс.

6. Что такое состояние

Обычная переменная в компоненте не подходит для данных, которые должны менять интерфейс.

Рассмотрим неправильный пример:

```
javascript
function Counter() {
  let count = 0

  function handleClick() {
    count = count + 1
  }

  return (
    <button onClick={handleClick}>
      Count is {count}
    </button>
  )
}
```

Кажется, что при клике `count` должен увеличиваться. Но интерфейс не обновится так, как ожидается.

Почему?

- обычная переменная не сохраняется между рендерами компонента;
- изменение обычной переменной не сообщает React, что интерфейс нужно обновить.

React должен знать: данные изменились, нужно заново отрисовать компонент.

Для этого используется состояние.

text

Состояние - это данные компонента, изменение которых приводит к обновлению интерфейса.

7. Что такое `useState`

`useState` - это хук React для добавления состояния в функциональный компонент.

Хук - это специальная функция React, имя которой начинается с `use`.

Чтобы использовать `useState`, его нужно импортировать:

javascript

```
import { useState } from 'react'
```

В нашем проекте это уже есть в начале `App.jsx`:

javascript

```
import { useState } from 'react'
```

Затем внутри компонента вызывается:

javascript

```
const [count, setCount] = useState(0)
```

Эта строка создает состояние.

8. Разбор записи `const [count, setCount] = useState(0)`

Запись выглядит непривычно для новичков, поэтому разберем ее медленно.

javascript

```
const [count, setCount] = useState(0)
```

`useState(0)` возвращает пару значений:

text

1. Текущее значение состояния.
2. Функцию для изменения этого состояния.

В нашем случае:

text

```
count - текущее значение счетчика
setCount - функция, которая меняет count
0 - начальное значение count
```

Можно представить так:

javascript

```
const result = useState(0)
const count = result[0]
const setCount = result[1]
```

Но в React обычно используют короткую запись через деструктуризацию массива:

```
javascript
const [count, setCount] = useState(0)
```

Правило именования:

```
text
const [чтоХраним, setЧтоХраним] = useState(начальноеЗначение)
```

Примеры:

```
javascript
const [age, setAge] = useState(18)
const [name, setName] = useState('Анна')
const [isOpen, setIsOpen] = useState(false)
```

9. Как обновляется состояние

Чтобы изменить состояние, нельзя просто написать:

```
javascript
count = count + 1
```

Так делать нельзя, потому что `count` объявлен через `const`, и потому что React не узнает об изменении.

Нужно использовать функцию обновления:

```
javascript
setCount(count + 1)
```

После вызова `setCount` React:

1. сохраняет новое значение состояния;
2. заново вызывает компонент;
3. получает новый JSX;
4. обновляет нужную часть интерфейса в браузере.

Цепочка:

```
text
Пользователь нажал кнопку
↓
Сработал onClick
↓
Вызвался setCount
↓
React обновил состояние
↓
React заново отрисовал компонент
↓
Пользователь увидел новое число
```

10. Почему в проекте используется `(count) => count + 1`

В нашем проекте написано так:

```
javascript
setCount((count) => count + 1)
```

Это называется обновление по предыдущему состоянию.

Функция:

```
javascript
(count) => count + 1
```

получает предыдущее значение `count` и возвращает новое.

Например:

text

```
было 0 -> стало 1
было 1 -> стало 2
было 2 -> стало 3
```

Можно было бы написать проще:

javascript

```
setCount(count + 1)
```

Для одного простого клика это обычно тоже будет работать. Но вариант:

javascript

```
setCount((count) => count + 1)
```

надежнее, когда новое значение зависит от предыдущего.

Особенно это важно, если обновлений несколько подряд:

javascript

```
setCount((count) => count + 1)
setCount((count) => count + 1)
setCount((count) => count + 1)
```

Так React сможет корректно применить каждое обновление по очереди.

11. Два варианта записи обработчика

Вариант 1. Обработчик прямо в JSX

javascript

```
<button onClick={() => setCount((count) => count + 1)}>
  Count is {count}
</button>
```

Плюс: коротко.

Минус: если логика станет длиннее, JSX будет сложнее читать.

Вариант 2. Отдельная функция

javascript

```
function App() {
  const [count, setCount] = useState(0)

  function handleCounterClick() {
    setCount((count) => count + 1)
  }

  return (
    <button onClick={handleCounterClick}>
      Count is {count}
    </button>
  )
}
```

Плюс: понятное имя функции. Код легче читать и объяснять.

Отдельная функция часто удобнее, потому что помогает увидеть структуру:

text

состояние -> обработчик -> JSX

12. Правила использования хуков

useState - это хук. У хуков есть важные правила.

Правило 1. Вызывать хук только на верхнем уровне компонента

Правильно:

javascript

```
function App() {  
  const [count, setCount] = useState(0)  
  
  return <button>Count is {count}</button>  
}
```

Неправильно:

javascript

```
function App() {  
  if (true) {  
    const [count, setCount] = useState(0)  
  }  
  
  return <button>Count</button>  
}
```

Хуки нельзя вызывать внутри if, циклов и вложенных функций.

Правило 2. Вызывать хук только внутри React-компонента или другого хука

Неправильно:

javascript

```
const [count, setCount] = useState(0)  
  
function App() {  
  return <button>Count is {count}</button>  
}
```

Правильно:

javascript

```
function App() {  
  const [count, setCount] = useState(0)  
  
  return <button>Count is {count}</button>  
}
```

На первых этапах достаточно запомнить:

text

useState пишем внутри компонента, до return, не внутри условий и циклов.

13. Состояние может быть разного типа

В `useState` можно хранить не только числа.

Число

```
javascript
const [count, setCount] = useState(0)
```

Строка

```
javascript
const [name, setName] = useState('Анна')
```

Логическое значение

```
javascript
const [isOpen, setIsOpen] = useState(false)
```

Массив

```
javascript
const [items, setItems] = useState([])
```

Объект

```
javascript
const [user, setUser] = useState({
  name: 'Анна',
  age: 18,
})
```

14. Пример: кнопка сброса счетчика

Добавим вторую кнопку, которая сбрасывает счетчик в ноль.

```
javascript
function App() {
  const [count, setCount] = useState(0)

  function handleIncrement() {
    setCount((count) => count + 1)
  }

  function handleReset() {
    setCount(0)
  }

  return (
    <>
      <button type="button" className="counter" onClick={handleIncrement}>
        Count is {count}
      </button>

      <button type="button" onClick={handleReset}>
        Reset
      </button>
    </>
  )
}
```

Здесь два обработчика:

```
text
  handleIncrement - увеличивает count на 1
  handleReset - устанавливает count в 0
```

Когда мы пишем:

```
javascript
  setCount(0)
```

мы говорим React: новое значение состояния должно быть 0.

15. Пример: показать или скрыть текст

Состояние может хранить true или false.

```
javascript
function Hint() {
  const [isVisible, setIsVisible] = useState(false)

  function handleToggle() {
    setIsVisible((isVisible) => !isVisible)
  }

  return (
    <>
      <button type="button" onClick={handleToggle}>
        Показать или скрыть подсказку
      </button>

      {isVisible && (
        <p>
          Состояние помогает React помнить, что нужно показать на странице.
        </p>
      )}
    </>
  )
}
```

Разберем:

```
javascript
  const [isVisible, setIsVisible] = useState(false)
```

Изначально подсказка скрыта.

```
javascript
  setIsVisible((isVisible) => !isVisible)
```

Меняет значение на противоположное:

```
text
  false -> true
  true -> false
```

Фрагмент:

```
javascript
  {isVisible && (
    <p>...</p>
  )}
```

означает: показать абзац только если isVisible равно true.

Условный рендеринг мы еще будем изучать отдельно, но этот пример хорошо показывает связь состояния и интерфейса.

16. Состояние локально для компонента

Состояние принадлежит тому компоненту, где оно создано.

Пример:

```
javascript
function Counter() {
  const [count, setCount] = useState(0)

  return (
    <button onClick={() => setCount((count) => count + 1)}>
      Count is {count}
    </button>
  )
}

function App() {
  return (
    <>
      <Counter />
      <Counter />
    </>
  )
}
```

На странице появятся две кнопки. У каждой будет свой независимый `count`.

Если нажать первую кнопку, изменится только первая. Если нажать вторую, изменится только вторая.

Это значит:

```
text
Состояние локально для конкретного экземпляра компонента.
```

17. Почему React не меняет страницу напрямую

В обычном JavaScript можно было бы найти элемент и изменить его текст:

```
javascript
document.querySelector('button').textContent = 'Count is 1'
```

В React обычно так не делают.

В React мы описываем, каким должен быть интерфейс при текущем состоянии:

```
javascript
<button>
  Count is {count}
</button>
```

А затем меняем состояние:

```
javascript
setCount((count) => count + 1)
```

React сам решает, как обновить браузер.

Главная идея:

text

Мы не командуем браузеру вручную менять текст.
Мы меняем состояние, а React обновляет интерфейс.

18. Частые ошибки новичков

Ошибка 1. Забыть импортировать `useState`

Неправильно:

javascript

```
function App() {  
  const [count, setCount] = useState(0)  
}
```

Правильно:

javascript

```
import { useState } from 'react'  
  
function App() {  
  const [count, setCount] = useState(0)  
}
```

Ошибка 2. Вызвать обработчик сразу

Неправильно:

javascript

```
<button onClick={handleClick()}>  
  Нажми  
</button>
```

Правильно:

javascript

```
<button onClick={handleClick}>  
  Нажми  
</button>
```

Или:

javascript

```
<button onClick={() => handleClick()}>  
  Нажми  
</button>
```

Ошибка 3. Менять состояние напрямую

Неправильно:

javascript

```
count = count + 1
```

Правильно:

javascript

```
setCount((count) => count + 1)
```

Ошибка 4. Вызвать `useState` внутри условия

Неправильно:

```
javascript
function App() {
  if (count > 0) {
    const [message, setMessage] = useState('Привет')
  }
}
```

Правильно:

```
javascript
function App() {
  const [message, setMessage] = useState('Привет')
}
```

Ошибка 5. Ожидать мгновенного изменения переменной после `setCount`

Новички иногда пишут:

```
javascript
function handleClick() {
  setCount(count + 1)
  console.log(count)
}
```

И удивляются, что в консоли может быть старое значение.

Важно: состояние в React обновляется для следующего рендера. Не нужно строить логику так, будто `count` изменится прямо в этой же строке.

Если новое значение зависит от старого, используйте функцию:

```
javascript
setCount((count) => count + 1)
```

19. Практика

Задание 1

В `src/App.jsx` найдите строку:

```
javascript
const [count, setCount] = useState(0)
```

Объясните письменно:

- что означает `count`;
- что означает `setCount`;
- что означает `0`.

Задание 2

Вынесите обработчик клика в отдельную функцию.

Было:

```
javascript
<button
  type="button"
  className="counter"
  onClick={() => setCount((count) => count + 1)}
>
  Count is {count}
</button>
```

Сделайте:

```
javascript
function handleCounterClick() {
  setCount((count) => count + 1)
}
```

и:

```
javascript
<button
  type="button"
  className="counter"
  onClick={handleCounterClick}
>
  Count is {count}
</button>
```

Задание 3

Добавьте кнопку сброса счетчика:

```
javascript
function handleResetClick() {
  setCount(0)
}
```

JSX:

```
javascript
<button type="button" onClick={handleResetClick}>
  Reset
</button>
```

Проверьте, что после нажатия на `Reset` счетчик снова показывает 0.

Задание 4

Добавьте новое состояние:

```
javascript
const [isHintVisible, setIsHintVisible] = useState(false)
```

Добавьте обработчик:

```
javascript
function handleHintClick() {
  setIsHintVisible((isHintVisible) => !isHintVisible)
}
```

Добавьте кнопку:

```
javascript
<button type="button" onClick={handleHintClick}>
  Toggle hint
</button>
```

Добавьте условный вывод текста:

```
javascript
{isHintVisible && (
  <p>useState хранит данные, которые влияют на интерфейс.</p>
)}
```

Задание 5

Запустите проект:

```
bash
npm run dev
```

Проверьте:

- счетчик увеличивается;
- кнопка `Reset` сбрасывает значение;
- кнопка `Toggle hint` показывает и скрывает подсказку.

20. Контрольные вопросы

5. Что такое событие в интерфейсе?
6. Как в React записывается обработчик клика?
7. Почему нужно писать `onClick={handleClick}`, а не `onClick={handleClick()}?`
8. Что такое состояние компонента?
9. Почему обычной переменной недостаточно для обновления интерфейса?
10. Что такое `useState`?
11. Что возвращает `useState`?
12. Что означает запись `const [count, setCount] = useState(0)?`
13. Что делает `setCount`?
14. Почему после `setCount` React обновляет интерфейс?
15. Почему `useState` нужно вызывать до `return`?
16. Можно ли вызывать `useState` внутри `if`?
17. Какие типы данных можно хранить в состоянии?
18. Что значит "состояние локально для компонента"?

19. Почему вариант `setCount((count) => count + 1)` надежен для обновления по предыдущему значению?

21. Краткий итог

События позволяют React-приложению реагировать на действия пользователя. Состояние позволяет компоненту помнить данные между рендерами. Хук `useState` создает пару: текущее значение и функцию для его обновления.

Главная цепочка интерактивности:

text

```
graph TD
    A[Пользователь делает действие] --> B[Срабатывает событие]
    B --> C[Обработчик вызывает set-функцию]
    C --> D[React обновляет состояние]
    D --> E[React заново отображает компонент]
```

Главная запись занятия:

javascript

```
const [count, setCount] = useState(0)
```